



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

Aula 21 - Estilo Arquitetural

Cliente x Servidor e Orientado a Serviços (SOA)

Prof.: Filipe Nhimi

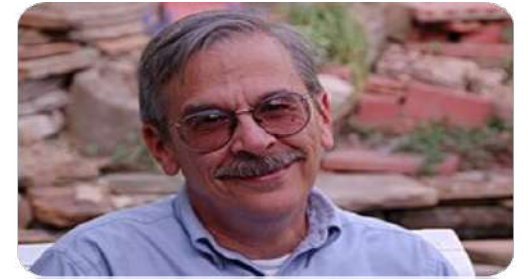
Agenda

- ❑ Conceitos
- ❑ Exemplos
- ❑ Considerações
- ❑ Conclusão



Arquitetura *Client x Server*

“Segundo Bass, Clements e Kazman, a arquitetura **cliente-servidor** é um estilo arquitetural no qual o sistema é estruturado em **dois tipos principais de componentes**: **clientes**, que solicitam serviços, e **servidores**, que fornecem esses serviços por meio de uma rede.”



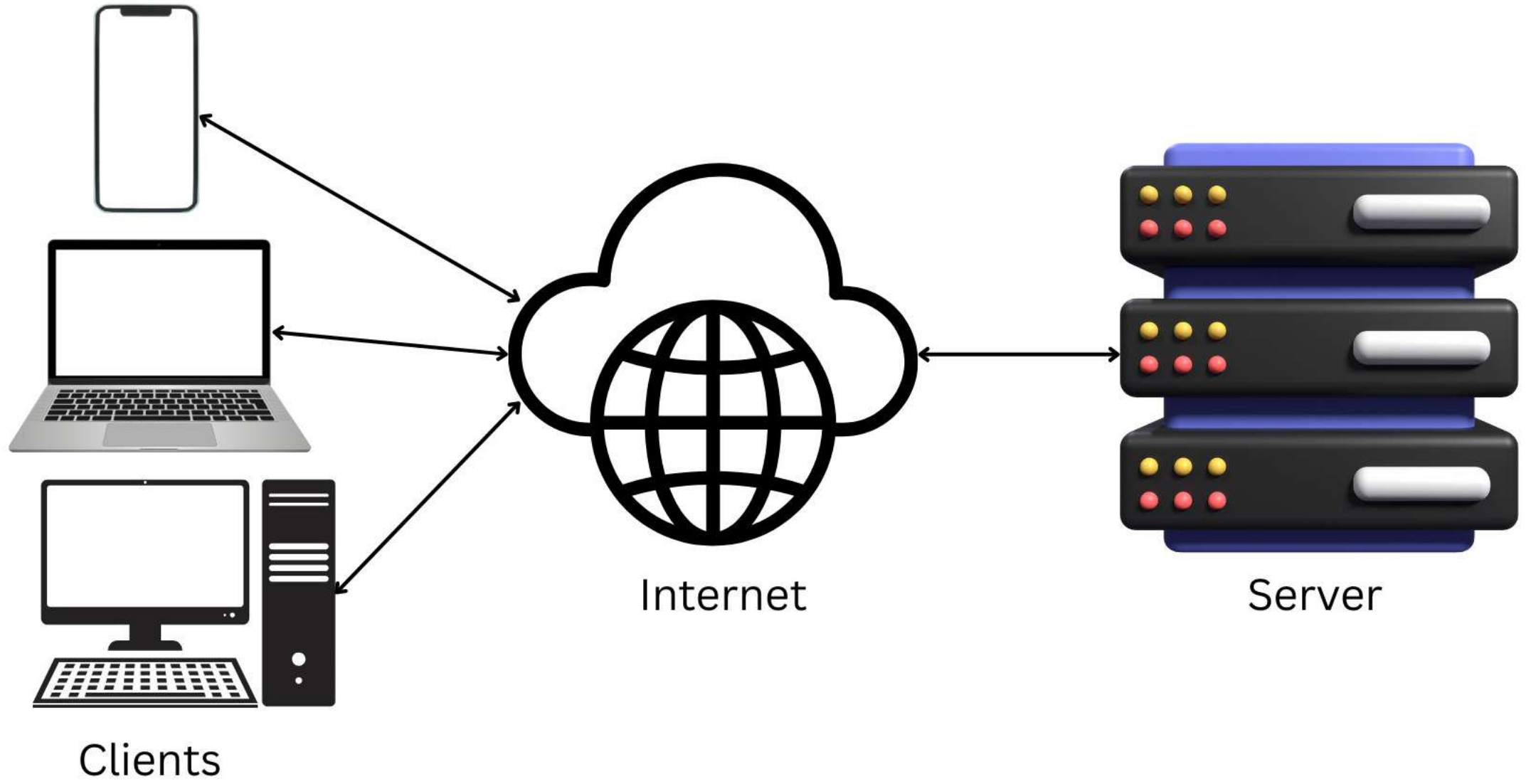
Len Bass



Paul Clements



Rick Kazman



Quais sistemas web utilizam o
Estilo

Cliente x Servidor?

Todos os sistemas web utilizam o modelo cliente-servidor como base de comunicação, embora possa combinar outros estilos arquiteturais mais complexos internamente.

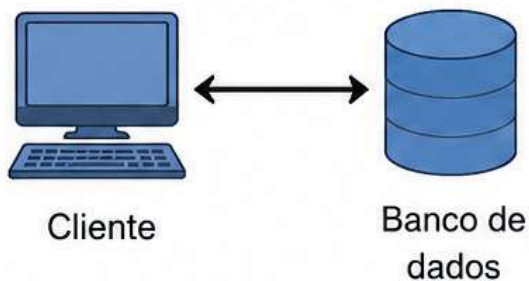
Cliente-servidor é a base da maioria das aplicações modernas.

Mesmo arquiteturas mais avançadas
(como *microservices*) ainda utilizam esse
modelo como fundamento

Variações do estilo

2-tier

Cliente acessa diretamente o banco de dados.

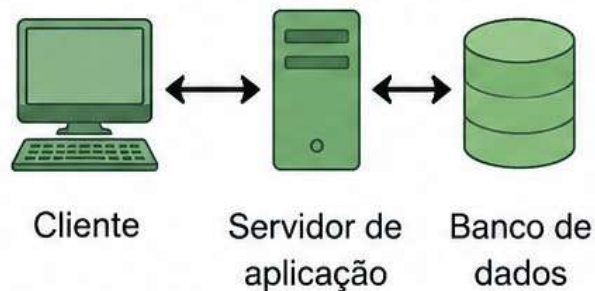


Exemplo:

Aplicações desktop antigas que acessam o banco diretamente.

3-tier

Cliente se comunica com o servidor de aplicação, que acessa o banco de dados.

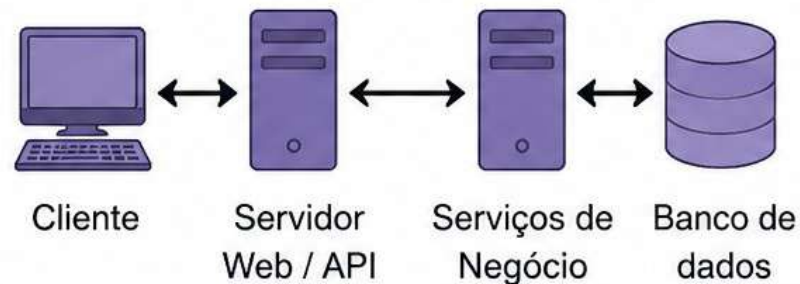


Exemplo:

Aplicações web com lógica de negócio no servidor de aplicação.

N-tier

Múltiplas camadas de servidores (e.g., API, serviços, banco de dados) para maior escalabilidade e separação de responsabilidades.



Exemplo:

Aplicações corporativas complexas com múltiplos serviços e camadas de persistência.

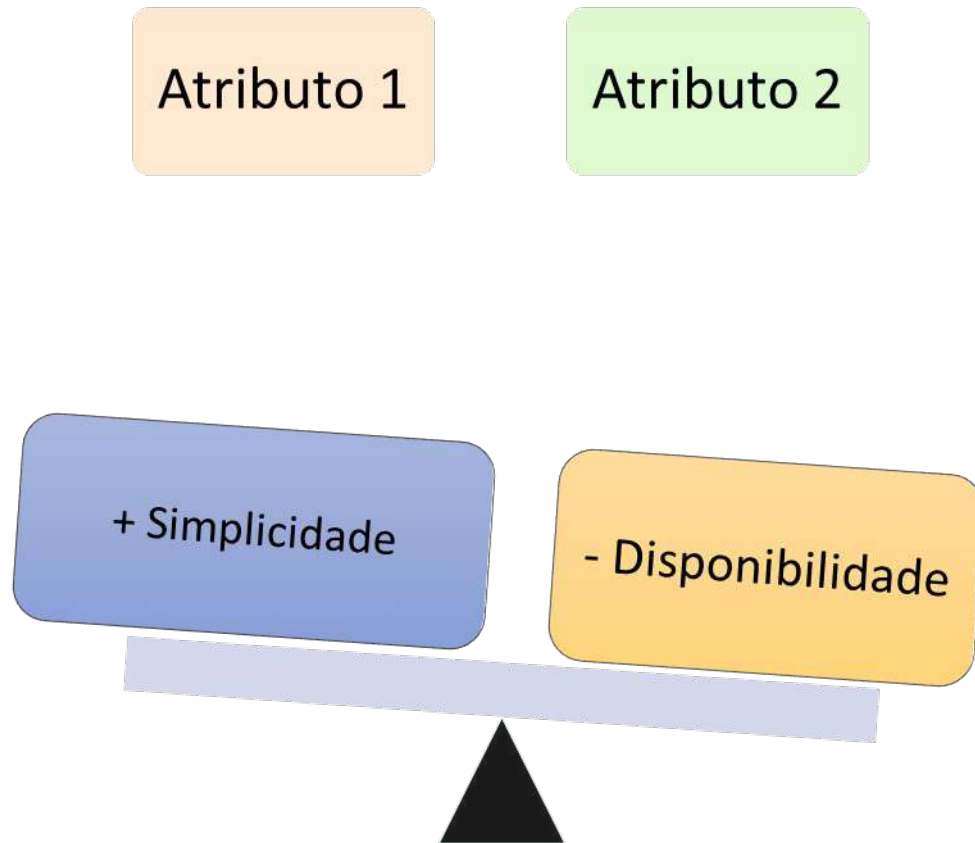
Vantagens (Cliente x Servidor)

- ❑ Centralização de dados e regras de negócio
- ❑ Facilidade de manutenção e atualização no servidor
- ❑ Maior controle de segurança
- ❑ Clientes mais simples
- ❑ Consistência de dados facilitada
- ❑ Facilidade de gerenciamento e monitoramento

Desvantagens (Cliente x Servidor)

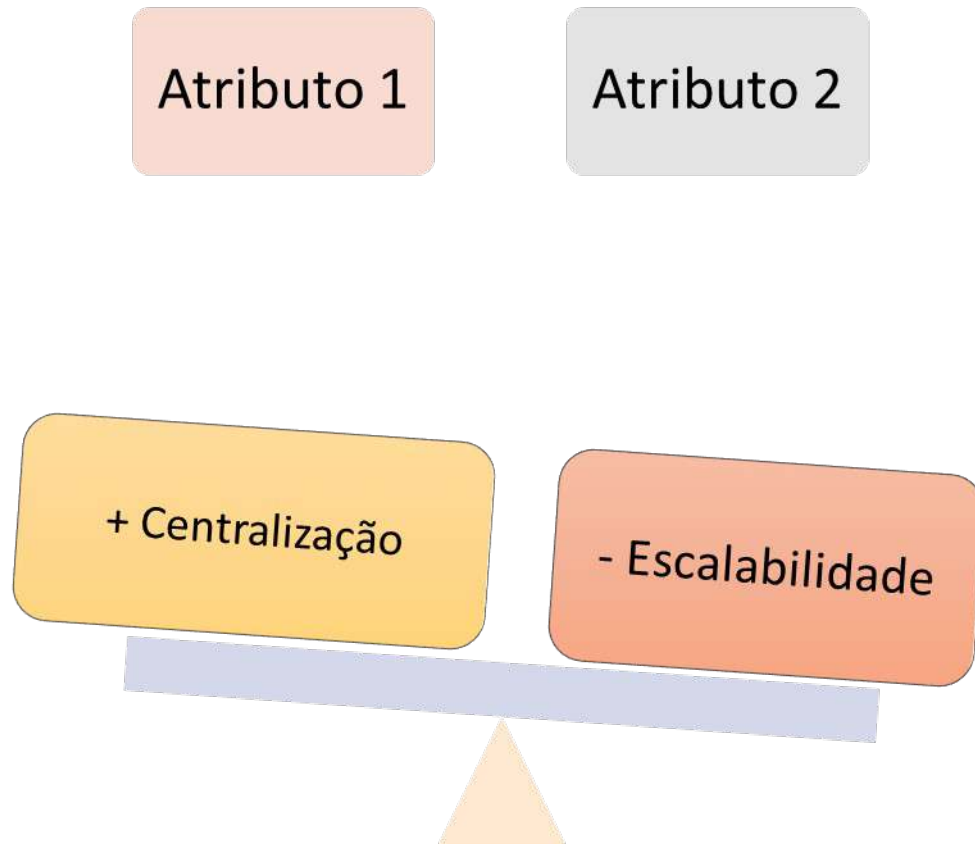
- ❑ Ponto único de falha no servidor
- ❑ Possível gargalo de desempenho
- ❑ Dependência da rede (latência e disponibilidade)
- ❑ Escalabilidade limitada sem mecanismos adicionais
- ❑ Maior carga de processamento no servidor
- ❑ Necessidade de infraestrutura para alta disponibilidade

Trade-offs do Estilo Cliente x Servidor



Servidor pode se tornar o **ponto único de falha** da arquitetura

Trade-offs do Estilo Cliente x Servidor



Centralizar lógica e dados no servidor simplifica controle e manutenção, porém, **pode virar gargalo** (servidor sobrecarregado). Para escalar será necessário replicação, balanceamento, etc.

S

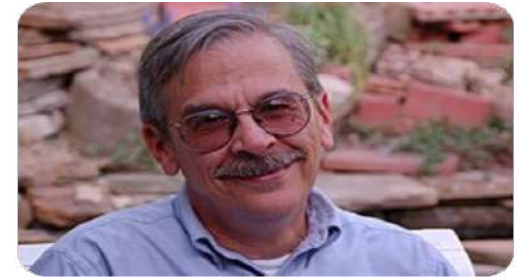
O

A

Service
Oriented
Architecture

Arquitetura Orientada a Serviços - SOA

“Segundo Bass, Clements e Kazman, a Arquitetura Orientada a Serviços (SOA) é um estilo arquitetural no qual a funcionalidade do sistema é organizada como um **conjunto de serviços interoperáveis, fracamente acoplados e acessíveis** por meio de interfaces bem definidas.”



Len Bass



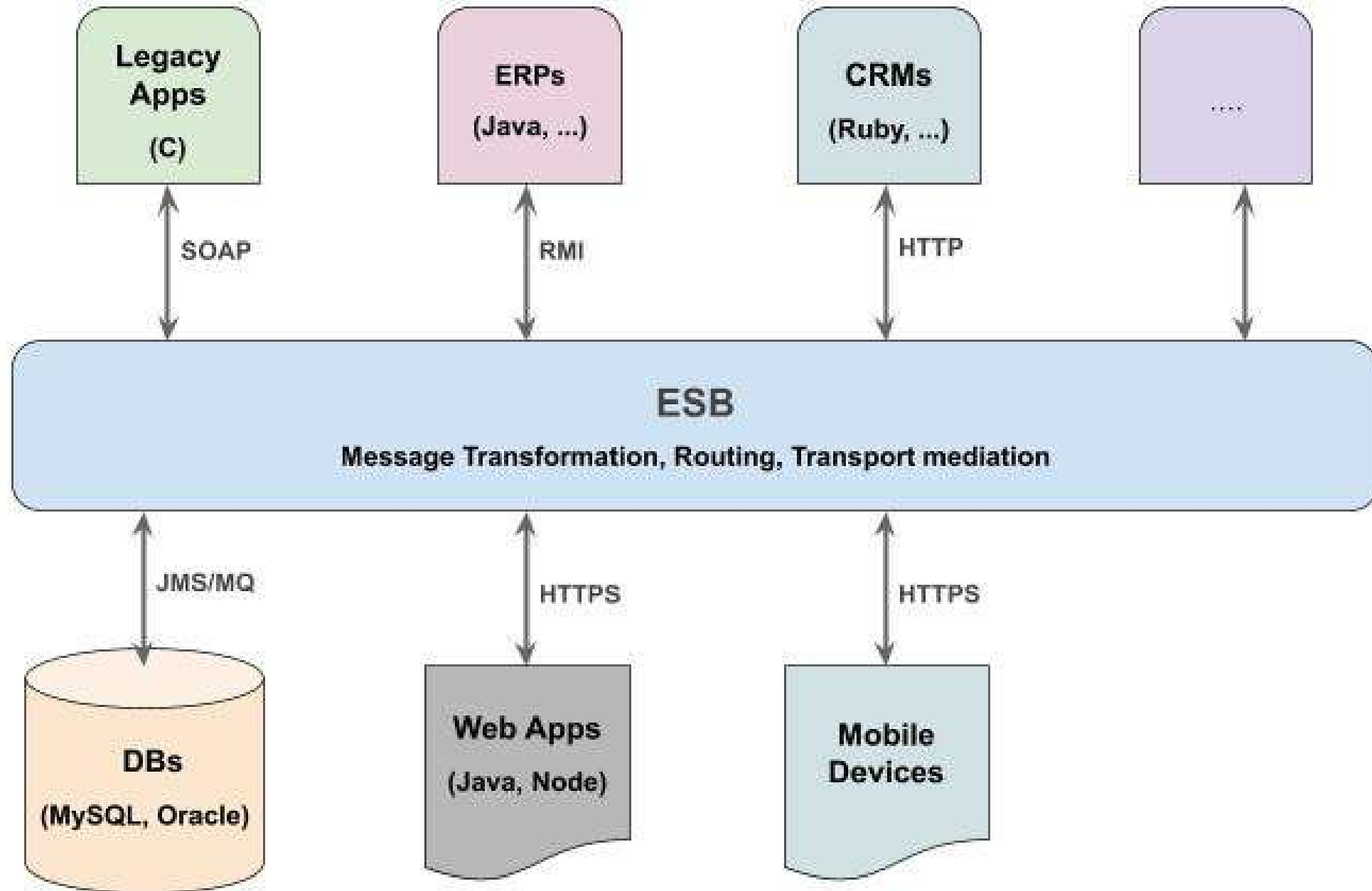
Paul Clements



Rick Kazman

Organizar aplicações em
torno de **serviços**
autônomos, fracamente
acoplados e padronizados...

Estilo arquitetural que se consolidou como uma das principais abordagens para promover **interoperabilidade entre sistemas...**





O ESB (Enterprise Service Bus) era o coração da **integração** no SOA tradicional.

Um middleware de integração que fica entre sistemas/serviços.

Funções:

- ☐ Roteamento
- ☐ Transformação (XML, SOAP, etc.)
- ☐ Mediação de protocolos
- ☐ Segurança
- ☐ Orquestração leve (em alguns casos)

O problema do ESB clássico:

- ☐ Centralização excessiva (“God ESB”)
- ☐ Gargalo de performance
- ☐ Acoplamento forte
- ☐ Dificuldade de evolução
- ☐ **Resultado:** O mercado começou a quebrar essas responsabilidades

O que o ESB *não* é?

Não é uma API que expõe endpoints para o cliente

Não é um back-end de domínio (Pedido, Pagamento etc.)

ESB x Web Service ou API

Característica	ESB	Web Service ou API
Papel	Integração	Expor lógica de negócio
Quem consome	Outros serviços / sistemas	Clientes (web/mobile)
Posição	Central	Borda (Domínio)
Lógica de negócio	Evitar	Aplicar

ESB não é um *back-end* de negócio; é uma infraestrutura de integração que intermedia a comunicação entre serviços.

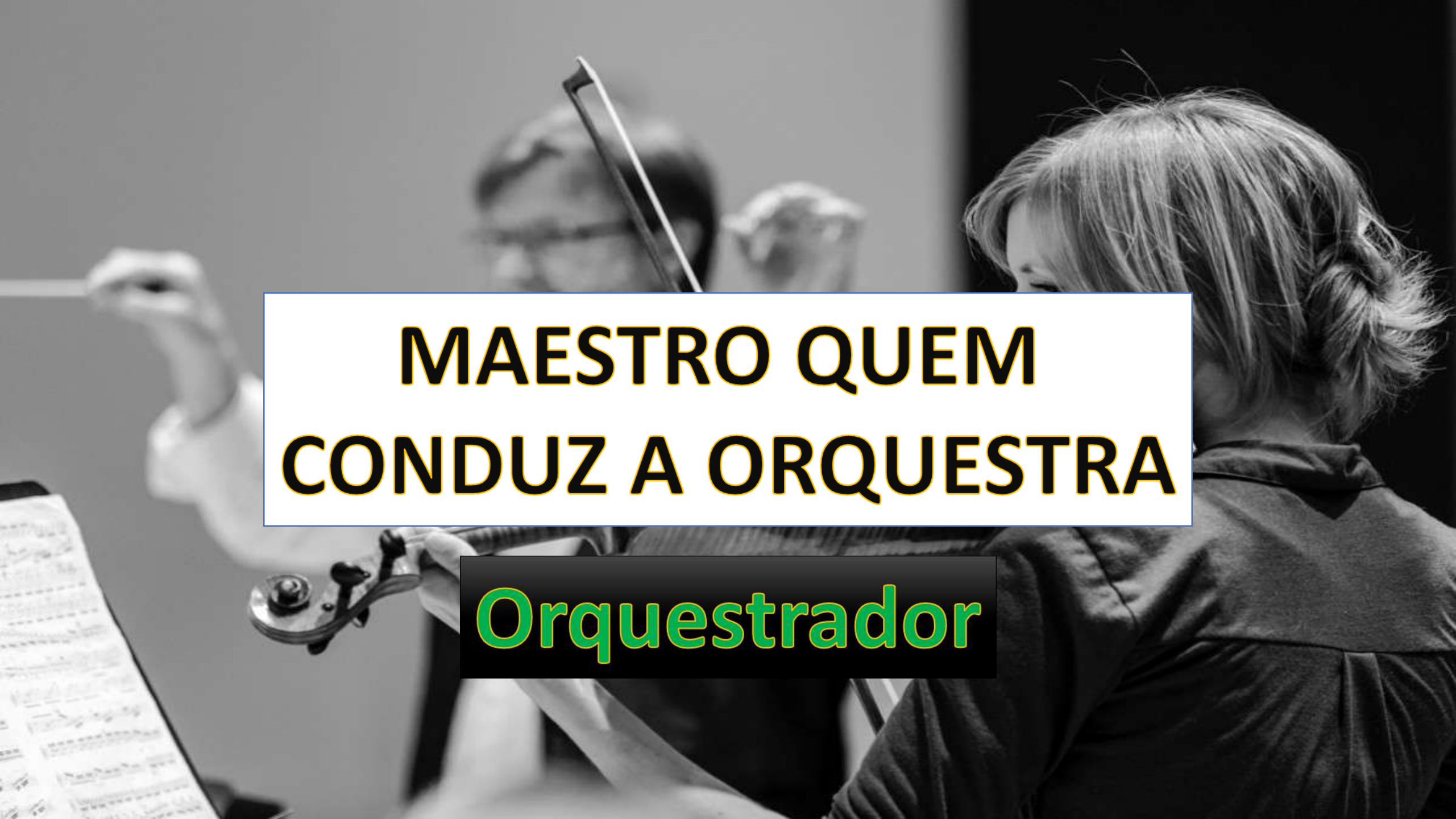
ESB



O ESB é uma secretária executiva altamente técnica, fala em vários idiomas **mas que não toma decisões estratégicas**, apenas viabiliza a comunicação de forma inteligente.

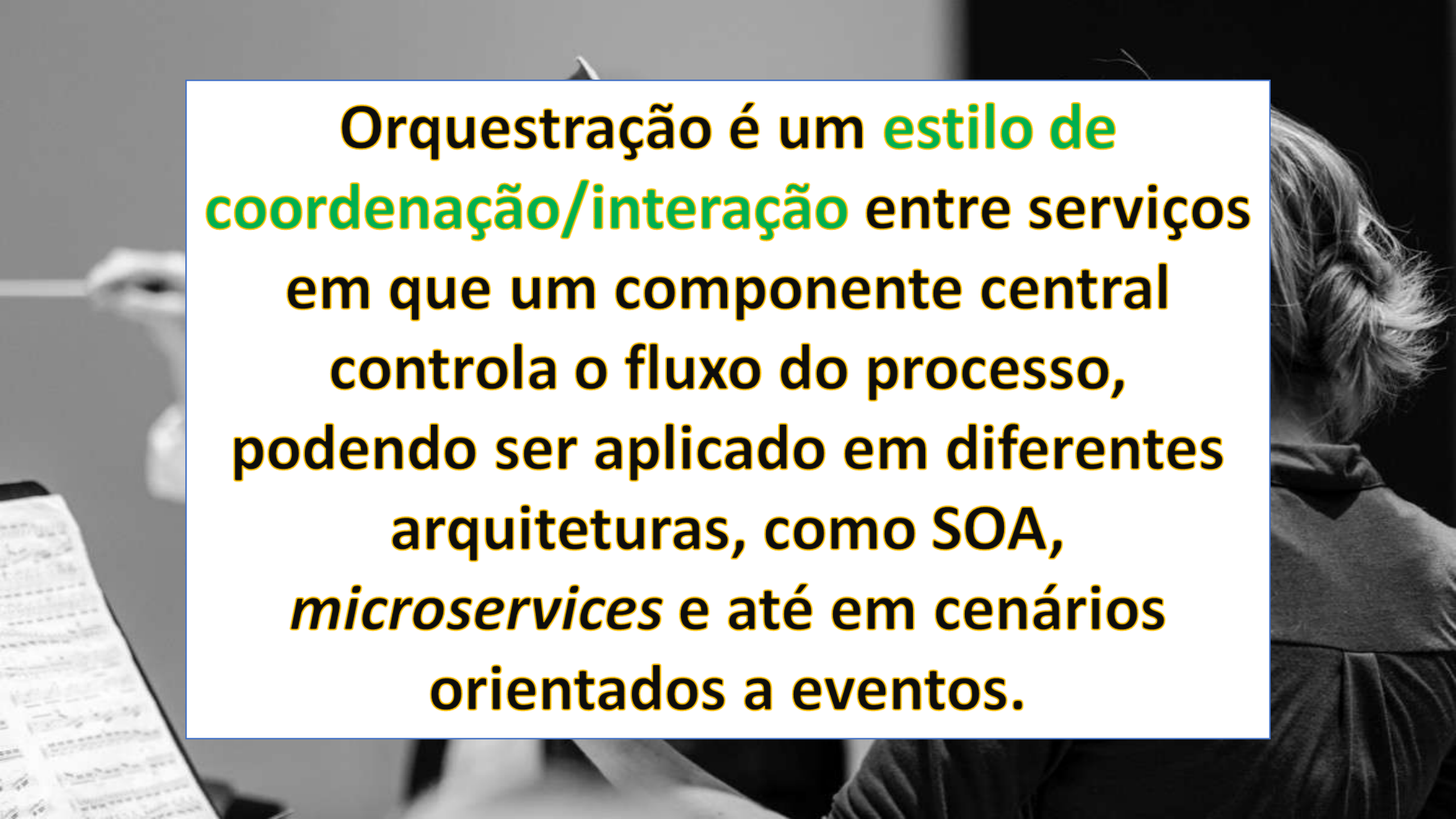
Se a Secretária é o ESB e **não**
toma decisões estratégicas
(fluxo de negócio)....

Então, quem faz isso?



MAESTRO QUEM CONDUZ A ORQUESTRA

Orquestrador



Orquestração é um **estilo de coordenação/interação** entre serviços em que um componente central controla o fluxo do processo, podendo ser aplicado em diferentes arquiteturas, como SOA, *microservices* e até em cenários orientados a eventos.

Fluxo de Chamadas – Orquestrador + ESB

O **Orquestrador** controla o fluxo de negócio.
O **ESB** viabiliza a comunicação entre os serviços.



Em que o ESB se transformou (na prática)?

Comunicação entre serviços

- ☐ Mensageria / eventos
- ☐ Substitui o “barramento” do ESB

Hoje, o que o ESB fazia virou papel dos API Gateway:

- ☐ Authentication
- ☐ Authorization
- ☐ Routing
- ☐ Rate limiting
- ☐ Caching
- ☐ Request Transformation

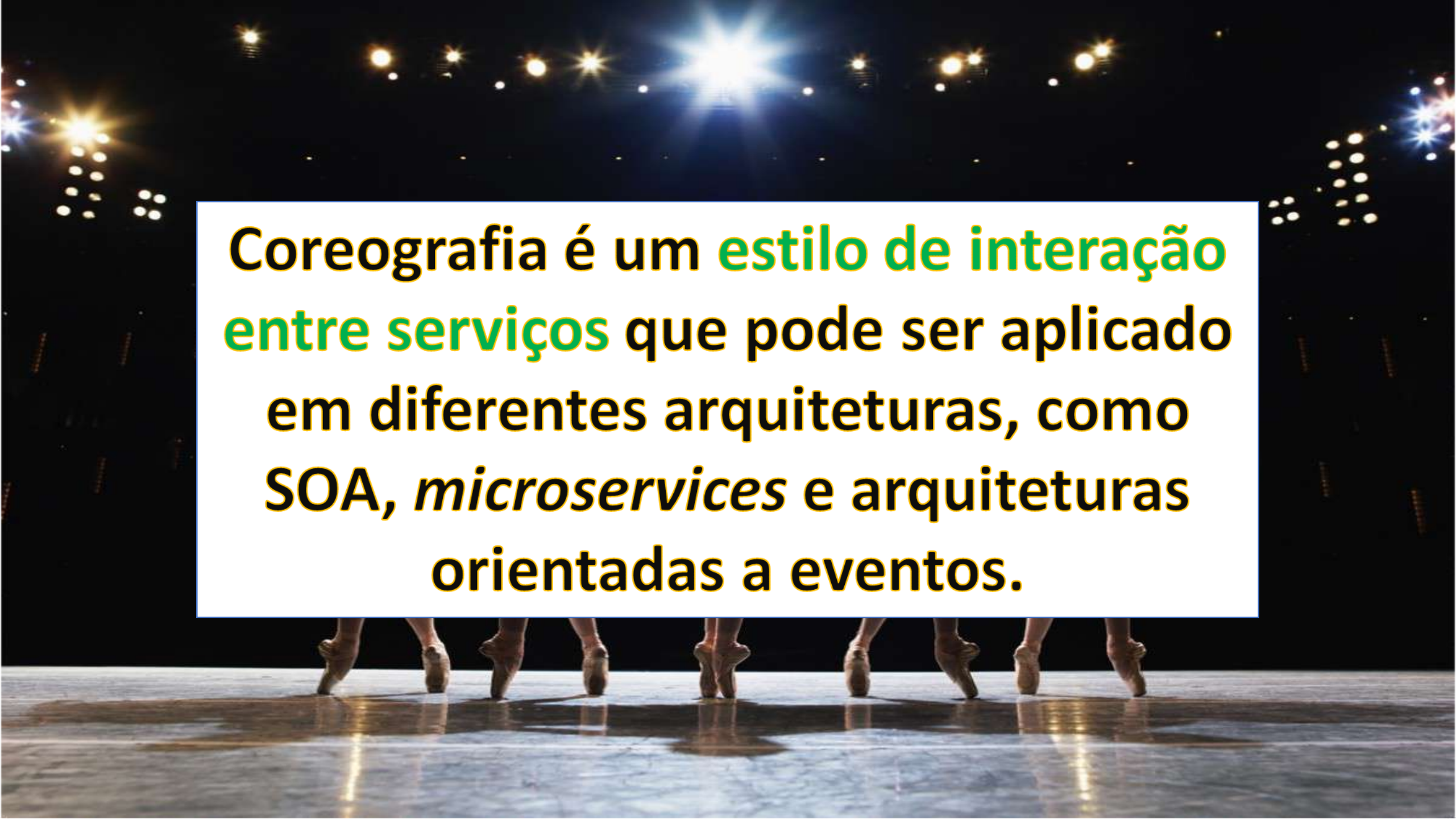


TECNOLOGIA PROVEU ISSO!

NOTA IMPORTANTE:

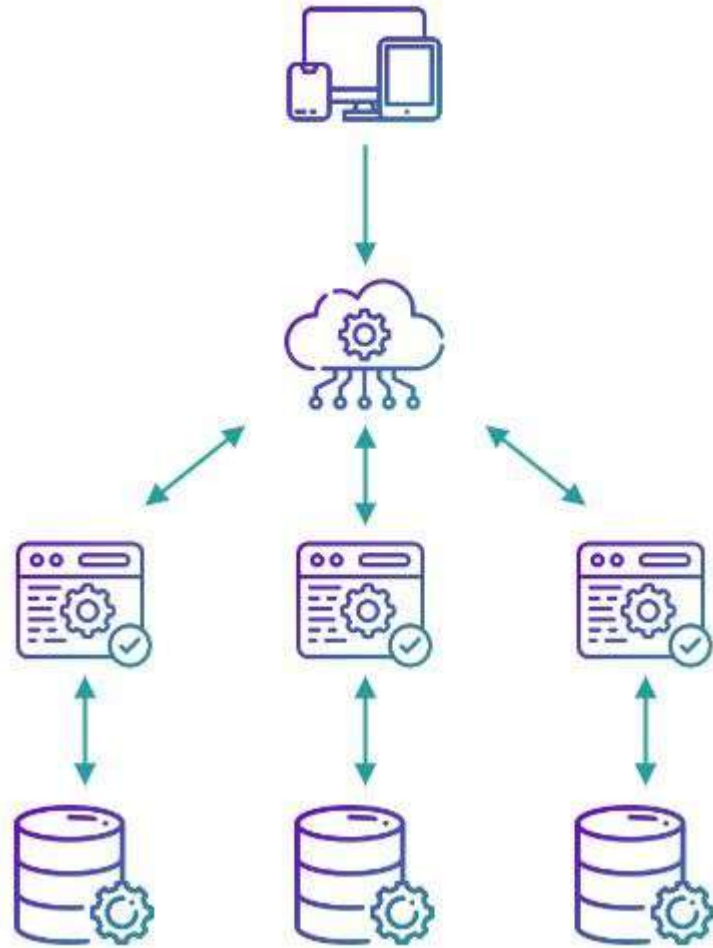
Service-Oriented Architecture

Pode usar Orquestração ou
Coreografia que são formas de
coordenação de fluxos de negócio

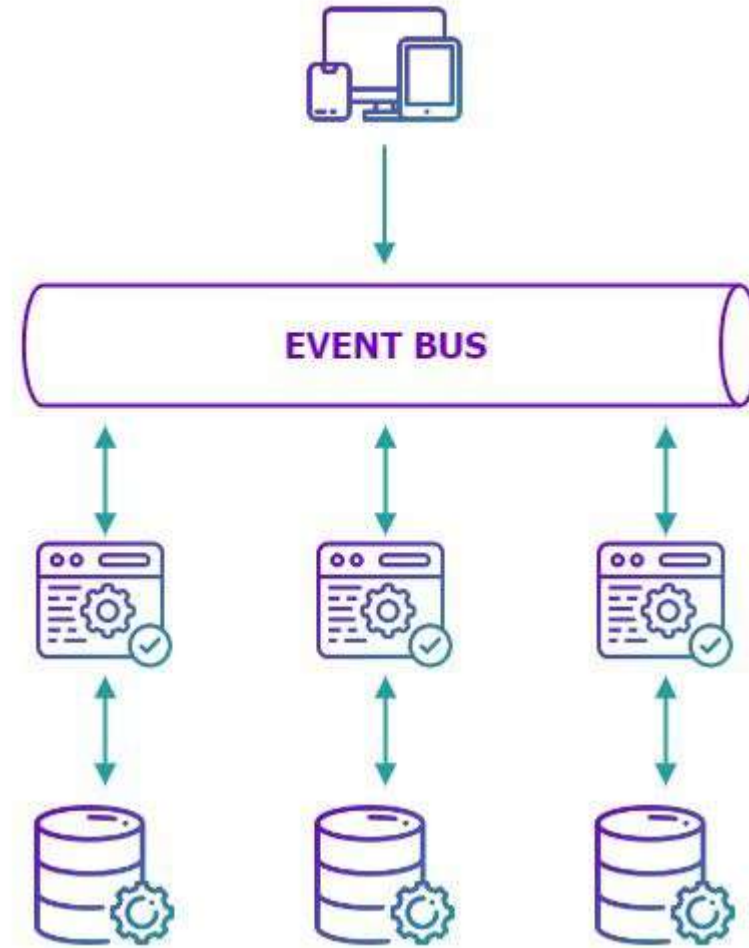
The background of the slide is a photograph of a stage. At the top, several bright spotlights create a starburst effect against a dark sky. In the foreground, the lower legs and feet of several dancers in pointe shoes are visible, standing on a reflective floor that shows their reflections. A white rectangular box is centered in the middle of the image, containing text.

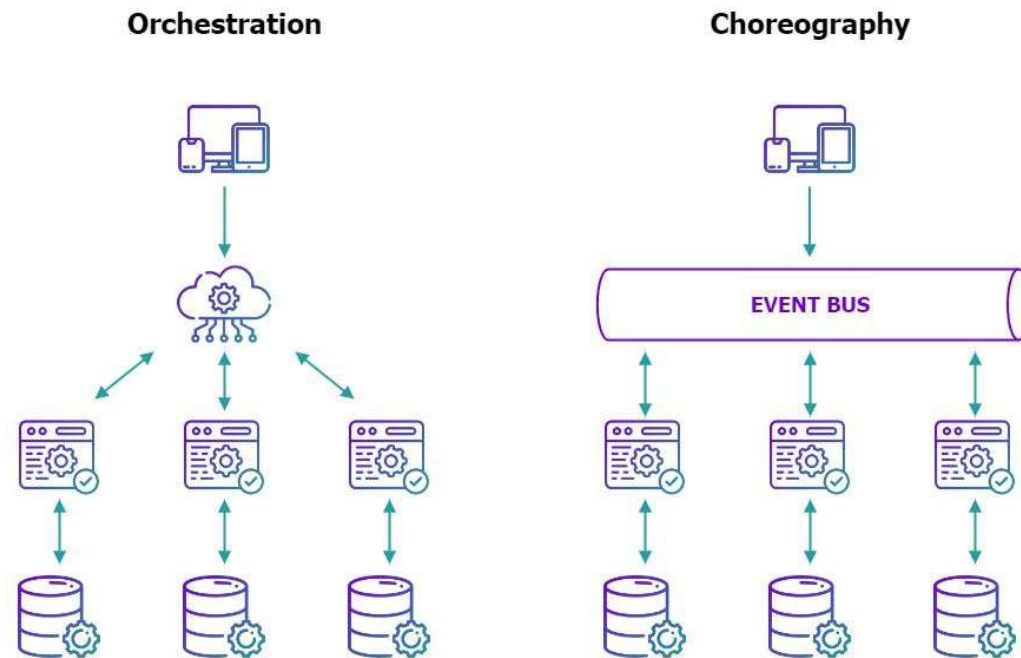
Coreografia é um **estilo de interação entre serviços** que pode ser aplicado em diferentes arquiteturas, como *SOA, microservices* e arquiteturas orientadas a eventos.

Orchestration



Choreography





Aspecto	Orquestração	Coreografia
Controle	Centralizado	Distribuído
Fluxo	Explícito	Implícito
Acoplamento	Médio	Baixo
Complexidade	Mais simples de entender	Mais difícil de rastrear

Exemplo real

Coreografia:

- ❑ Eventos simples (pedido criado, pagamento aprovado)

Orquestração:

- ❑ Fluxos críticos (cancelamento, rollback, **saga**)

Orquestração privilegia controle!

Coreografia privilegia autonomia!

SAGA Pattern

*Em Sistemas Distribuídos **NÃO dá para usar** controle de transação **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade), **pois não existe banco de dados único.***

Como garantir consistência entre transações realizadas por vários serviços?

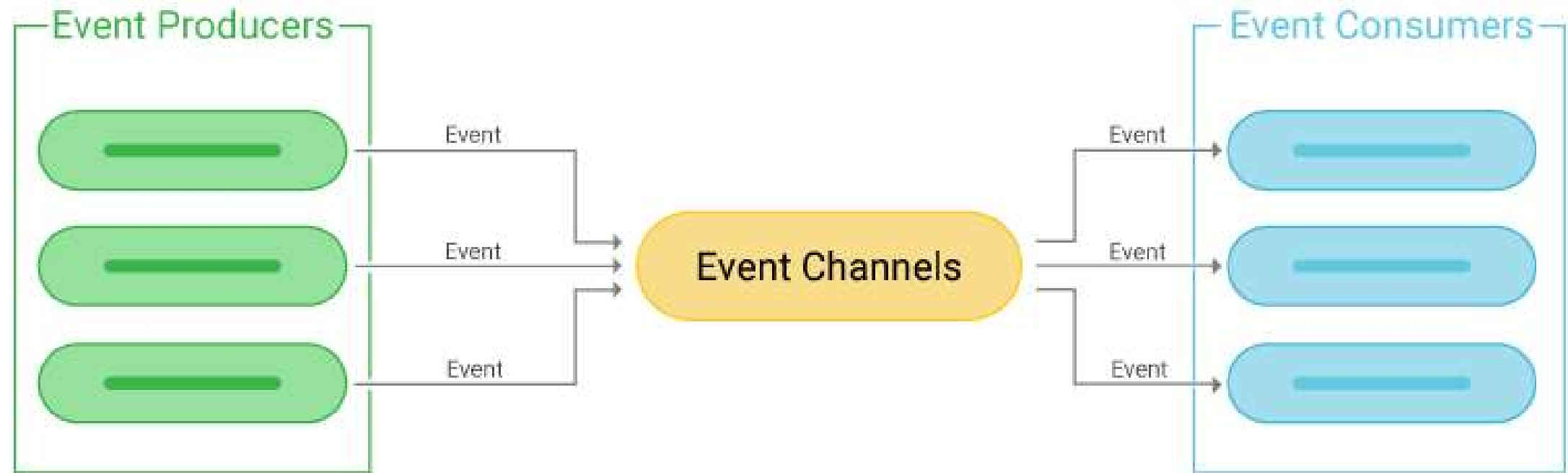
SAGA Pattern

What is **SAGA** Design Pattern?

Saga Pattern is a sequence of Local Transactions where each transaction has a **compensating action** for rollback in case of failure.



Evolução do SOA



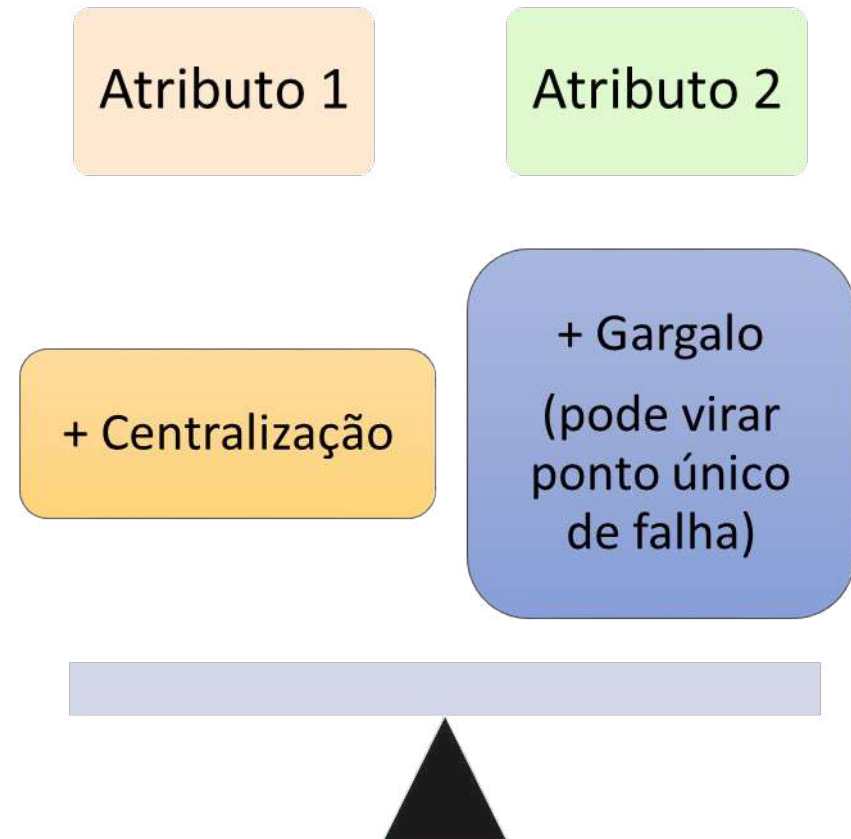
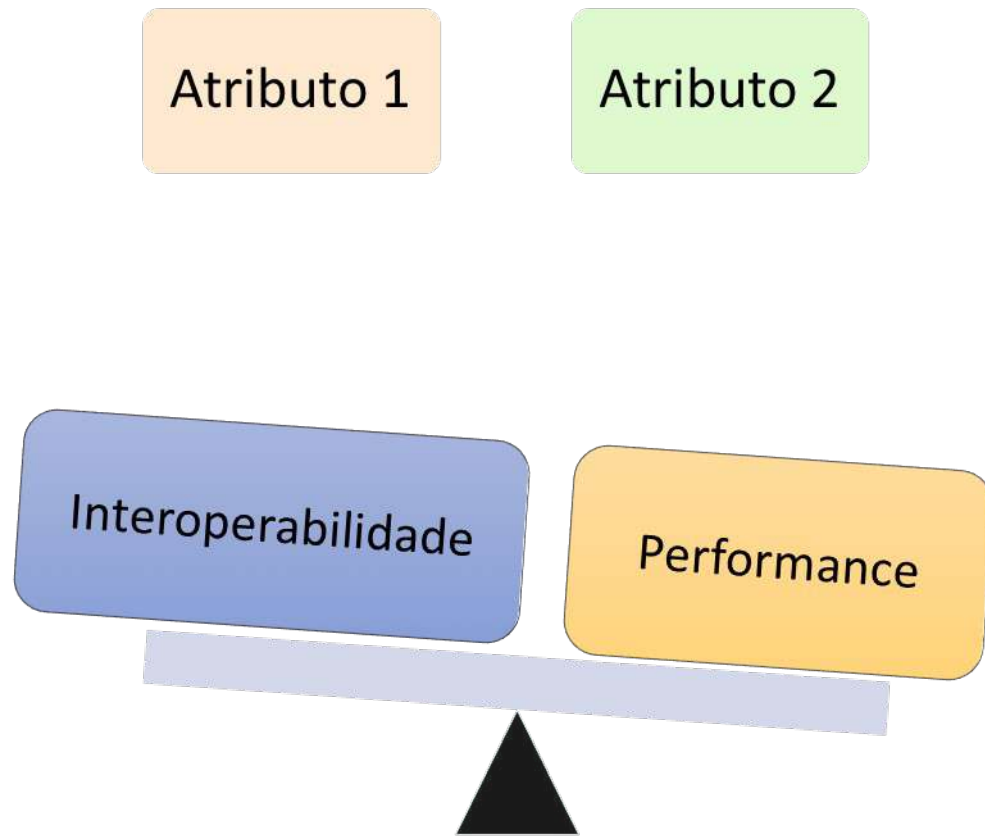
Vantagens (Orientação a Serviços - SOA)

- ❑ Reuso de serviços
- ❑ Baixo acoplamento
- ❑ Interoperabilidade
- ❑ Centralização de capacidades
- ❑ Alinhamento com negócio

Desvantagens (Orientação a Serviços - SOA)

- ❑ Complexidade arquitetural
- ❑ Dependência de infraestrutura (ex: ESB)
 - ❑ Pode virar gargalo quando “God ESB”
- ❑ Overhead de comunicação
- ❑ Governança pesada
 - ❑ Versionamento
 - ❑ Contratos
 - ❑ Políticas
- ❑ Dificuldade de testes

Trade-offs do Estilo Orientado a Serviços (SOA)



Exemplos de Utilização (Orientado a Serviços - SOA)

☐ **Sistemas bancários**

- ☐ Integra conta, cartão, empréstimos e pagamentos
- ☐ Alto reuso e interoperabilidade
- ☐ Forte necessidade de governança e segurança

☐ **ERP corporativo**

- ☐ Integra módulos como financeiro, RH e estoque
- ☐ Reuso de regras de negócio
- ☐ Pode gerar acoplamento indireto entre áreas

☐ **Sistemas hospitalares**

- ☐ Integra prontuário, laboratório e faturamento
- ☐ Evita duplicidade de dados
- ☐ Integração complexa com sistemas legados

☐ **Logística e transporte**

- ☐ Integra pedidos, rastreamento e transportadoras
- ☐ Visão unificada do processo
- ☐ Dependência de integrações externas

☐ **Sistemas governamentais (ex: GOV.BR)**

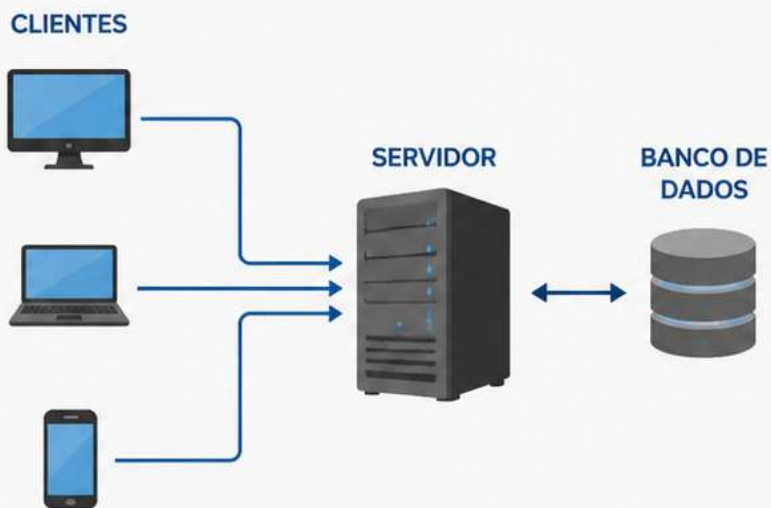
- ☐ Integra autenticação, assinatura e dados públicos
- ☐ Alta interoperabilidade entre órgãos
- ☐ Alta complexidade e dependência de múltiplos sistemas

SOA é mais indicado em ambientes grandes, heterogêneos e com forte necessidade de integração

Relação/Comparação

CLIENTE x SERVIDOR

Modelo tradicional com centralização da lógica e dos dados no servidor.



- Comunicação: requisição / resposta
- Centralização da lógica de negócio
- Mais simples de desenvolver inicialmente
- Escalabilidade limitada (servidor como gargalo)

ORIENTADA A SERVIÇOS (SOA)

Sistema composto por serviços independentes que se comunicam para entregar valor.



- Serviços independentes e reutilizáveis
- Integração entre sistemas e organizações
- Maior flexibilidade e escalabilidade
- Comunicação via serviços (APIs / mensagens)

Conclusão

SOA organiza sistemas como um conjunto de **serviços interoperáveis** e **fracamente acoplados**, promovendo **reuso, padronização e alinhamento com o negócio** para **resolver problemas de integração** em larga escala, ao custo de maior complexidade arquitetural, necessidade de governança rigorosa e possíveis impactos de desempenho, cujos princípios seguem influenciando arquiteturas modernas.